Exercises · Vector Databases and Search

AI04 Exercises

Work each one before reading its solution. Exercises 1 to 3 and 5 are arithmetic. Practise Exercise 6 out loud.

Exercise 1: Size an index

You have 60 million chunks at 1536 dimensions. Serving nodes have 64 GB of usable memory. An HNSW graph uses about 264 bytes per vector.

(a) Compute the float32 vector size. **(b)** Add the graph. How many nodes do you need? **(c)** Repeat for `halfvec` and for int8. **(d)** Repeat for float32 at 768 dimensions. **(e)** What would you test before choosing?

Solution

(a) 60M × 1536 × 4 = **368.64 GB**.

(b) Graph: 60M × 264 = 15.84 GB. Total 384.48 GB, which needs **7 nodes**.

(c) `halfvec`: 184.32 + 15.84 = 200.16 GB, or **4 nodes**. int8: 92.16 + 15.84 = 108 GB, or **2 nodes**.

(d) 184.32 + 15.84 = 200.16 GB, or **4 nodes**, the same as `halfvec` at the full dimension.

(e) Recall@10 at a fixed latency on your own queries, including filtered ones, for each option. Node count alone doesn't settle it.

Exercise 2: Correlated filters

A tenant holds 10% of the data, in 10 of 100 topic clusters. Candidates per search: 40.

(a) If the filter were independent, what's the chance that all 40 candidates fail? **(b)** Assume instead that a query's candidates all come from its own cluster. What share of queries from other tenants' topics returns nothing? **(c)** So why would this tenant's own users rarely see this? **(d)** Then who does get hurt, and how?

Solution

(a) 0.9^{40} = **1.5%**.

(b) Close to **100%**, since candidates lie entirely outside the tenant's clusters.

(c) Their queries are usually about their own topics, so candidates land in their own clusters, and match. So tests with typical queries can look fine.

(d) Users asking about something the tenant covers thinly or in unusual words. Their query lands in other tenants' regions, and gets nothing, although relevant data exists. It's especially bad for smaller tenants, whose vectors are outnumbered in every region. This is why tenants need partitions.

Exercise 3: RRF by hand

Using `k = 60`:

Document	BM25 rank	Vector rank
A	1	Absent
B	4	2
C	Absent	1
D	2	3

(a) Compute each RRF score to five decimals, and rank them. **(b)** What does the result say about agreement?

Solution

(a) A = 1/61 = 0.01639. B = 1/64 + 1/62 = 0.03175. C = 1/61 = 0.01639. D = 1/62 + 1/63 = 0.03200. Order: **D, B, then A and C tied.**

(b) D and B appear in both lists, and each scores about twice what a list-topping document found by only one method scores. RRF rewards agreement between independent methods, as HLD M21A explains.

Exercise 4: Find the flaws

"Our vector store handles 8,000 queries per second at 3 ms. We tested with 1 million random vectors, no filters, reads only, using the default search settings, and reported mean latency."

List at least five problems, and say what you'd do instead.

Solution

1. **No recall figure.** Fix a recall target, and compare latency there.
2. **Random vectors.** Real data has structure. Use your embeddings.
3. **No filters.** Include rare and correlated ones.
4. **Reads only.** Test with concurrent writes.
5. **Mean latency.** Report p99.
6. **Default settings.** Sweep them, as defaults may have poor recall.
7. **Small scale.** Test near your expected size, including when the index exceeds memory.

Exercise 5: Plan tiers

Tenants hold these chunk counts: one at 30M, four at 5M, 50 at 200K, and 5,000 at 4K. Each chunk uses 1,288 bytes in RAM (int8 at 1024 dimensions, plus graph). Nodes have 90 GB usable.

(a) Compute the total memory. **(b)** With a dedicated threshold of 1M chunks, how much memory is dedicated, and how much shared? **(c)** How many nodes at minimum, ignoring replicas? **(d)** Name one risk that the arithmetic doesn't show.

Solution

(a) Chunks: 30M + 20M + 10M + 20M = 80M. × 1,288 B = **103.04 GB**.

(b) Dedicated: 50M × 1,288 B = **64.4 GB**. Shared: 30M × 1,288 B = **38.64 GB**.

(c) 103.04 / 90 rounds up to **2 nodes**, for example one with the dedicated tenants, and one shared.

(d) Query load. A small tenant with heavy traffic can be noisier than a large, quiet one, so promotion should consider query rate as well as size. Also, one node failure affects many tenants without replicas.

Exercise 6: Defend a decision

Your architect says: "Let's use one big index with a `tenant_id` filter. It's simple, and vendors say filtering is fast now." You have 5,000 tenants. Write the answer you'd give out loud.

Model answer

"Simplicity matters, and the filter approach works for many small, similar tenants, so I'm not rejecting it outright. Some engines do filter well during the walk.

But I've got two concerns. First, quality: tenants correlate with content, and in a test, a tenant with 10% of data got empty results on 88% of queries using post-filtering. So I'd want filter-aware search with a tenant-aware storage layout, and not simply a filter. Second, safety: with one index, a single missing filter is a cross-tenant leak.

So I'd keep the shared index for small tenants, using a tenant-organized layout, enforce the tenant from the token, and add an automated cross-tenant test. Large or regulated tenants get dedicated indexes.

I'd change my mind if a benchmark on our real data, with our real tenant distribution, shows that a single filtered index meets our recall target for the smallest tenants too, and if security accepts the isolation model. That's a test we can run this sprint."

Why this works: it credits the argument, gives evidence, proposes a smaller alternative, and names the test that would change your mind.

Your notes